czii-cryo-et-object-identification

1位解法 [部分的なU-NETによるセグメンテーションとアンサンブルのパート]

Rank	1
Team	Daddies
Author	Dieter
Topic ID	561510
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561510
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

このエキサイティングなコンペを開催してくれた kaggle と関係者の皆さんに感謝します。とても良い学びの機会で、私たちのコンピュータビジョンの経験の多くが 3D イメージングにも応用できると分かったのは非常に興味深いことでした。すばらしいチーム体験をくれた @bloodaxe に感謝します。

TLDR

解法は、セグメンテーションモデル（ResNet と B3 エンコーダを持つ 3D Unet）と、MONAI の物体検出モデル（SegResNet と DynUnet backbone）のアンサンブルです。データ拡張にも MONAI を使い、モデルを jit または TensorRT でエクスポートしました。これにより 200% の高速化が得られ、アンサンブルを少し大きくできました。外部データやシミュレーションデータは一切使っていません！

この投稿ではセグメンテーションベースのアプローチとアンサンブルを扱います。物体検出パートについては @bloodaxe の記事 1st place solution [Object Detection Part] を参照してください。

交差検証

セグメンテーションのアプローチでは、experiment ごとに分割するだけの 7 fold を使いました。7 fold すべての f4-score の平均は、LB とかなり良い相関がありました。モデル学習中は、各エポックの終わりに、検証用 experiment での単純なグリッドサーチでクラスごとの閾値を最適化しました。7 fold すべての学習が終わった後は、OOF 予測を使って閾値を再調整できました。ある fold の閾値を、他の 6 fold の予測に当てはめて求めます。そして得られた f4 曲線を平均し、最良の閾値を選びます。

データの前処理 / データ拡張

3D 画像は標準的な正規化で処理しました。つまり、630x630x184 の各画像について、パッチに分割する前に平均を引き、標準偏差で割りました。

モデルはスクラッチから学習するので、過学習を防ぐためにデータ拡張が不可欠でした。

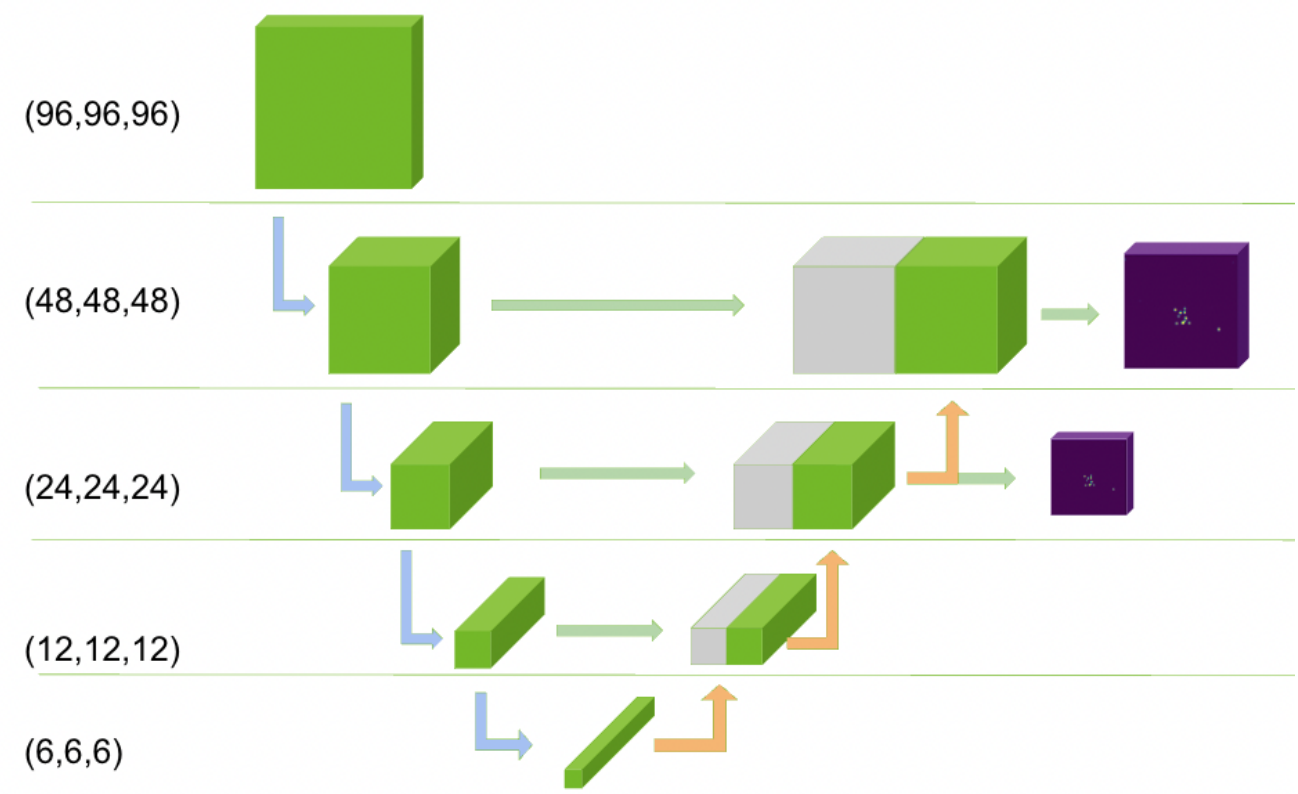
RandomCrop、各軸での Flip、回転を使いました。これらはすべて MONAI にあります。加えて、自前の MixUp 実装を使いました。これは、より長く学習して過学習を防ぐのにとても効果的でした。

モデル

このコンペでのモデリングは、なかなかの道のりでした。最初は、3D のガウスの球をセグメンテーションのターゲットにした単純な UNET から始めました。多様性のために MONAI の物体検出の例も試したところ、そのままでとてもよく機能することに気づきました。しかし、出力される特徴マップの違いを分析し、ガウスのヒートマップを使ったセグメンテーションに対する性能向上の要因を切り分けようとしたところ、どこに利点があるのかが分かったので、それに合わせてセグメンテーションモデルを調整しました。分かったのは次のことです。

- 最後から2番目の特徴マップの方が、最後の特徴マップより精度が高い。これは最初は意外でした
- box の回帰による利得は無視できるほど小さい。同じ種類の粒子は、だいたい同じ大きさだからです。

したがって、最後から2番目の特徴マップの出力に、ピクセル単位の loss をかけるだけで十分です。つまり、部分的な UNET を使うということです。背景のクラス重みを小さくして抑えれば、ガウスのヒートマップは不要で、単一のピクセルをターゲットにするだけで済みます。同じアプローチを低いレベルの出力にも使うこと（= deep supervision）は可能ですが、大きな利得はありません。セグメンテーションモデルの入力は 96x96x96 の画像パッチで、loss は 48x48x48 の出力で計算します。



全体として、比較的小さなモデルが本当によく機能し、loss の設計が最も重要な要素だと分かりました。MONAI の FlexibleUnet を、backbone の resnet34 と efficientnet-b3 で使いました。このアーキテクチャの6つのチェックポイントは2時間以内に推論が終わり、LB で7位のスコアになります。

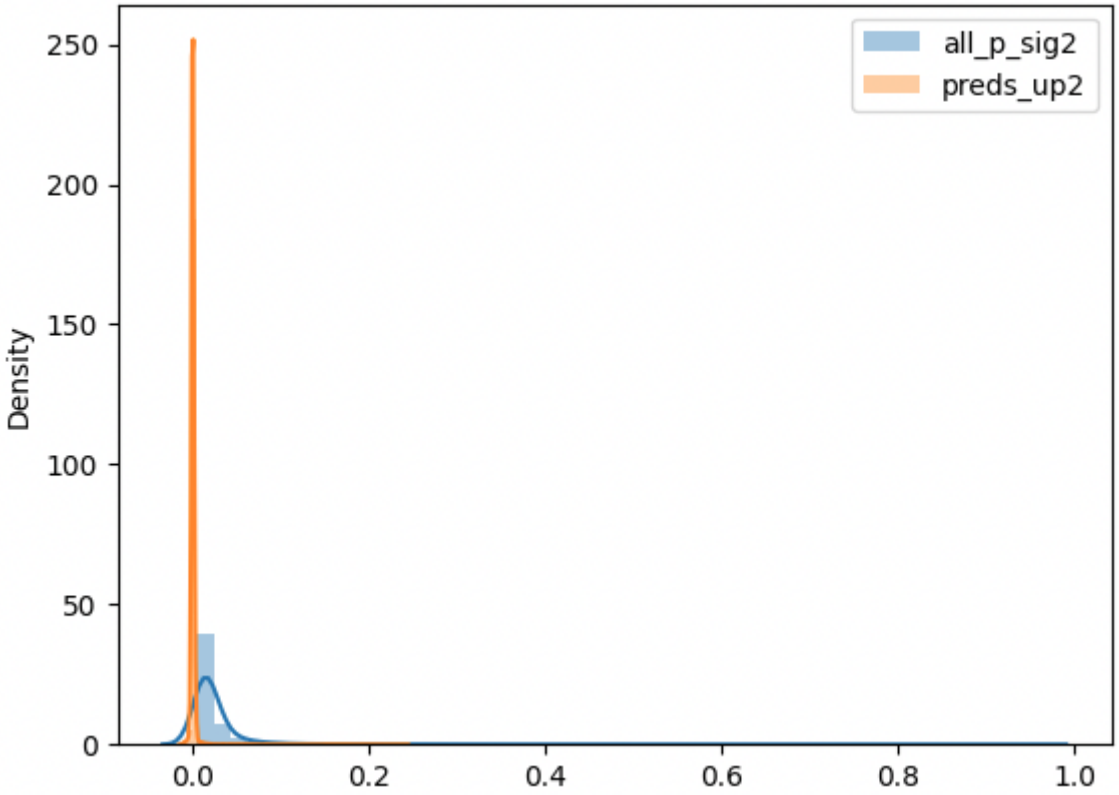
学習の手順

7クラス（背景を含む）と重み付き CrossEntropy を loss として使いました。特に、beta-amylase は採点されませんが、クラスとして残しておくのはかなり役立ちます。モデルが beta-galactosidase をそれと区別することを学ぶからです。陽性ピクセルの数が少ないことに対応するため、陽性ピクセルの重みを 256、背景の重みを 1 にしています。モデルは、ピーク LR 0.001 の cosine 学習率スケジュール、mixed precision、実効バッチサイズ 32 サンプルで学習しました。学習はランダムクロップに基づいており、検証では1つの experiment の画像をパッチに分割して RAM に保存しています。

アンサンブル

私たちの2つのアプローチは大きく異なるので、アンサンブルはとても難しいものでした。理論上は、物体検出の後処理を実行する前に、セグメンテーションモデルの予測と物体検出モデルの特徴マップ出力をアンサンブルでき

ます。しかし実際には、アーキテクチャと loss 関数の違いのため、それらの特徴マップの分布はまったく異なります。

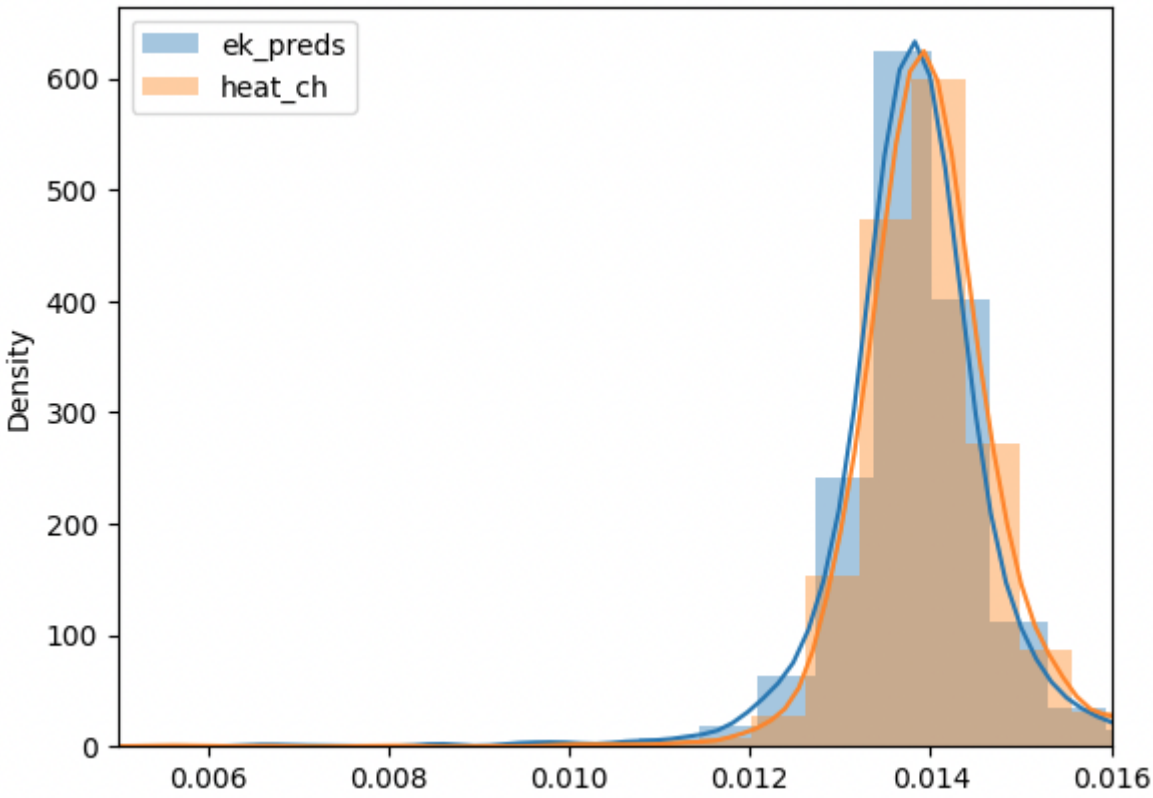


アンサンブルの可能性が見えていたので、このスケールの問題をエレガントに解決する方法を熱心に探しました。最良の提出のスケーリングは次のように機能します。予測 A と予測 B を組み合わせるために、各クラスについて A と B のすべてのピクセル値をソートし、B の値を、同じ順位にある A の対応する値で置き換えます。コードでは次のとおりです。

```
# rank scaling per class

adj_preds_B = torch.zeros_like(preds_B)
for c in range(n_classes):
    q = preds_B[c].flatten().argsort().argsort()
    r = preds_A[c].flatten()
    r2 = r.argsort()
    adj_preds_B[c] += r[r2[q]].reshape(preds_B[c].shape)
```

これにより、両方の予測が同じ分布になるので、物体検出のタスクを実行する前に、特徴マップを単純にブレンドできました。



効かなかったこと

- 補助データ（外部データまたはシミュレーションデータ）の使用
- その他のデータ拡張
- その他の loss（Tversky、Dice）

読んでいただきありがとうございました。

編集:

学習コード: https://github.com/ChristofHenkel/kaggle-cryoet-1st-place-segmentation

推論カーネル: https://www.kaggle.com/code/christofhenkel/cryo-et-1st-place-solution?scriptVersionId=223259615

補足Q&A

Ma Edward — 質問

おめでとうございます！！！

最終アンサンブルに入っている単体モデルの性能を共有していただけませんか。

Dieter — 回答

セグメンテーションについて:

resnet34 backbone: 0.766

resnet34 backbone + deep supervision: 0.767

effnet-b3 backbone: 0.765

組み合わせ: 0.778

Bartley — 質問

詳細な記事とすてきなアンサンブル戦略をありがとうございます。

あなたの MixUp は、通常の実装とどう違うのですか。

Đăng Nguyễn Hồng — 質問

おめでとうございます。いつもながら記事をありがとうございます！

CV のスキームは本当に印象的です。アンサンブルのアプローチは、ヒストグラムマッチングの一種のように見えますが、そうですか。

単体モデルについては、あなたのチームと上位の解法が、すべてをこれほどスムーズに機能させていることが驚きです。私は「似たように聞こえる」設定で、もっと低い結果しか出せず苦労しました。あなたたちにとっては自然に身についているけれど、私にとってはまだ少し難しい実装の細部がたくさんあるのでしょう。

私たちの問題点と失敗をよりよく理解できるよう、学習/推論コードを見られるのをとても楽しみにしています！

Yassine Alouini — 質問

お見事、おめでとうございます。学習インフラについて詳しく教えていただけますか（GPU の数、学習にかかった時間、loss に関する問題など）。

Dieter — 回答

他の実装と大きくは違いません。loss を混ぜる人もいますが、私はターゲットを混ぜる方が好きです。GPU 上でモデルの一部として持てるように、シンプルな純粋 torch の実装が好きです。また 1D、2D、3D に柔軟に対応できます。

```
torch
torch  nn
torch.nn.functional  F
torch.nn.parameter  Parameter
torch.distributions  Beta

(nn.Module):
  ():

        (Mixup, ).__init__()
        .beta_distribution = Beta(mix_beta, mix_beta)
        .mixadd = mixadd

  ():

        bs = X.shape[]
        n_dims = (X.shape)
        perm = torch.randperm(bs)
```

```
        coeffs = .beta_distribution.rsample(torch.Size((bs,))).to(X.device)
        X_coeffs = coeffs.view((-,) + (,)*(X.ndim-))
        Y_coeffs = coeffs.view((-,) + (,)*(Y.ndim-))

        X = X_coeffs * X + (-X_coeffs) * X[perm]

         .mixadd:
            Y = (Y + Y[perm]).clip(, )
    :
            Y = Y_coeffs * Y + ( - Y_coeffs) * Y[perm]

         Z:
             X, Y, Z

         X, Y


(nn.Module):

  ():
       (Net, ).__init__()

    …
       .mixup = Mixup(cfg.mixup_beta)
    …

  ():

       x = batch[]
       y = batch[]
        .training:
            torch.rand()[] < .cfg.mixup_p:
               x, y = .mixup(x,y)
    ….
```

Yassine Alouini — 質問

彼は既存の実装を 3D パッチで動くように改変したのだと思います。私の知る限り、MixUp は2枚の画像を重み alpha と (1-alpha) で組み合わせるものです。もしかすると、時間方向に画像を持たせるために何か賢いことをしたのでしょうか。

Dieter — 回答

6つのチェックポイントを学習しました。それぞれ A100 で bf16 の mixed-precision を使って2時間かかります。私のモデルにとって mixup は決定的に重要で、学習のエポック数は2倍以上になります。

Dieter — 回答

私が公開 LB で1位だったのは2か月前で、ガウスの球をターゲットにした単純な 3D-Unet でした。当時のスコアは約 0.74 でした。最終的な解法が、あなたの言うように「スムーズに」見えるところまで改善して磨き上げるのに、2か月の懸命な作業が必要でした。

Dieter — 回答

Mixup は、本当に、バッチの入力とターゲットを Beta 分布から引いた重みで平均するだけです。

実はこれは、4年前に（@philippsinger と @ilu000 と）、たしか rainforest コンペの一環としてコードを書いたものです。Mixup は音声に対して本当に強力です。私たちの実装は当時から任意の次元の入力に柔軟に対応し、GPU 上で動いていたので、それ以来いくつかのコンペで再利用しています。